

КОМПЛЕКСНАЯ САМОСТОЯТЕЛЬНАЯ РАБОТА ПО АДМИНИСТРИРОВАНИЮ LINUX-СЕРВЕРОВ

Цель работы: получить навыки настройки веб-инфраструктуры с балансировкой нагрузки, контейнеризации, мониторинга и централизованного сбора логов в рамках единого проекта.

Задание

Развернуть и настроить отказоустойчивый веб-сервис со следующим стеком технологий:

- веб-сервер с балансировкой нагрузки: frontend: Nginx, backend: Apache
- контейнеризации: Docker
- мониторинг инфраструктуры: Prometheus, Grafana
- централизованный сбор логов: ELK-стек (Elasticsearch, Logstash, Kibana)

Технические требования

ОС: Ubuntu 22.04 LTS / 24.04 LTS

Архитектура: рекомендуется использовать 2 виртуальные машины:

VM1: Веб-сервер + экспортеры (мин. 2 ГБ RAM, 2 ядра CPU, 15 ГБ диска)

VM2: Prometheus + Grafana + ELK (мин. 4 ГБ RAM, 2 ядра CPU, 25 ГБ диска)

Доступ в интернет: требуется для установки пакетов и Docker-образов

Пошаговая инструкция выполнения

1. Конфигурирование web-сервера (Apache, Nginx, балансировка Nginx)

- установите Nginx и Apache
- настройте работу Apache на прослушивание портов 8081, 8082, 8083
- создайте тестовые страницы для каждого порта (например, "Server 1", "Server 2", "Server 3")
- настройте работу Nginx на порт 80
- настройте upstream в Nginx с тремя бэкендами (порты 8081-8083)
- настройте виртуальный хост в Nginx с proxy_pass на upstream

Примечание: пример конфигурационных файлов приведен в прикрепленных файлах

Проверка работы:

Проверка балансировки (выполнить несколько раз)

curl http://localhost

Должны поочередно появляться Server 1, Server 2, Server 3

Задание считается выполненным, если

- установлены Nginx и Apache
- Apache слушает три разных порта (8081, 8082, 8083)
- Nginx настроен на порту 80 как балансировщик нагрузки
- запросы к порту 80 распределяются между тремя экземплярами Apache

Дополнительное задание:

- установите docker
- скачайте официальный образ Nginx
- запустите контейнер Nginx на базе образа Nginx на порту 8080 (остановите хостовый Nginx или используйте команду с пробросом порта: -p 8080:80)
- подключите конфигурационные файлы Nginx из основной части задания в контейнер Nginx через volume

Дополнительное задание считается выполненным, если

- контейнер nginx запущен и доступен на порту 8080
- конфигурация балансировки загружена в контейнер
- балансировка работает через контейнер

2. Мониторинг: настройка сервера Prometheus и сбор метрик веб-сервера

2.1 Установка и настройка Prometheus (на VM2):

- установите Prometheus
- настройте конфигурационный файл prometheus.yml
- запустите службу, проверьте доступность на порту 9090

2.2 Установка экспортеров на веб-сервер:

- установите node_exporter для сбора системных метрик (CPU, память, диск, сеть), порт 9100
- для Nginx: либо установите nginx-prometheus-exporter (порт 9113), либо настройте stub_status в конфигурации Nginx.
- для Apache: либо установите apache_exporter, либо настройте модуль mod_status. Убедитесь, что все экспортеры работают и доступны по соответствующим портам

2.3 Настройка сбора метрик в Prometheus

Добавьте в prometheus.yml target'ы для каждого экспортера:

- job_name: 'node'

static_configs:

- targets: ['IP_VM1:9100']

- job_name: 'nginx'

static_configs:

- targets: ['IP_VM1:9113'] # или nopr stub_status

Перезапустите Prometheus для применения изменений.

2.4 Настройка визуализации метрик (Grafana на VM2)

- установите Grafana
- добавьте Prometheus как источник данных
- импортируйте готовые дашборды:

для системных метрик: дашборд с ID 1860 (Node Exporter Full)

для Nginx: дашборд с ID 12708 (NGINX Prometheus Exporter)

для Apache: дашборд с ID 3894 (Apache Dashboard)

Дополнительное задание: создайте собственный дашборд, объединяющий ключевые метрики вашей инфраструктуры.

Задание считается выполненным, если:

- Prometheus доступен на порту 9090, targets в состоянии UP
- node_exporter и минимум один веб-экспортер установлены и работают
- Метрики отображаются в интерфейсе Prometheus
- Grafana отображает дашборды с актуальными данными

Проверка работы:

- проверка экспортеров:

curl http://localhost:9100/metrics # node_exporter

curl http://localhost:9113/metrics # nginx-exporter (если используется)

- проверка Prometheus:

откройте <http://ваш-сервер:9090/targets> - все targets должны быть в состоянии UP
введите в графе метрику up - значение должно быть 1 для всех экспортеров.

- проверка Grafana:

дашборды должны отображать актуальные данные и обновляться в реальном времени.

Рекомендации: для тестирования мониторинга создайте нагрузку на веб-сервер (например, с помощью ab или siege)

3. Логирование: настройка централизованный сбора логов в ELK

3.1 Развёртывание ELK через Docker Compose (на VM2):

- установите Docker Compose
- используйте готовый `docker-compose.yml` для ELK 8.x
- настройте Logstash для приёма логов Nginx

3.2 Настройка сбора логов:

- настройте Nginx на запись логов в JSON-формате
- настройте Filebeat для отправки логов в Logstash
- создайте индекс в Elasticsearch

3.3 Визуализация в Kibana

- создайте index pattern в Kibana
- постройте простые визуализации:
- количество запросов по часам
- статус-коды ответов
- самые популярные URL

Задание считается выполненным, если:

- ELK-стек развёрнут и доступен (Kibana на порту 5601)
- Логи Nginx поступают в Elasticsearch
- Создан index pattern в Kibana
- Построены минимум 3 визуализации и собран дашборд

Ссылки на документацию:

Nginx upstream: https://nginx.org/ru/docs/http/nginx_http_upstream_module.html

Prometheus exporters: <https://prometheus.io/docs/instrumenting/exporters/>

ELK Docker: <https://www.elastic.co/guide/en/elastic-stack-get-started/current/get-started-docker.html>